Servidor HTTP/1.1 com Qualidade de Serviço em C com Pthreads: terceira entrega

Giovana Piazza[1], **Luis Felipe Maccalli**[1], **Vitor Moreira**[1]

[1]Universidade Federal do Pampa (Unipampa)
Fundamentos e Avaliação de Redes de Computadores – 2026/2

Abstract. *This report presents the second version of a concurrent, persistent HTTP/1.1 server written in C using POSIX threads. It implements per-IP rate limits, admission control, shared rates among connections and a live terminal panel. The revision validates configuration and reports aggregate per-IP throughput. Original packet captures are retained separately from new automated tests, which include complete downloads of the images referenced by the HTML page. Evaluation using Wireshark and IPTraf and experiments on physical networks remain pending and are not reported as completed.*

Resumo. *Este relatório apresenta a segunda versão de um servidor HTTP/1.1 concorrente e persistente em C com Pthreads. A implementação acrescenta taxas por IP, controle de admissão, divisão da taxa entre conexões e painel em tempo real. A revisão valida a configuração e corrige a medição agregada por IP. As capturas originais são preservadas separadamente dos novos testes automatizados, que incluem downloads completos das imagens referenciadas no HTML. A avaliação com Wireshark e IPTraf e os experimentos em redes físicas permanecem pendentes, sem atribuição de resultados que não foram obtidos.*

1. Introdução

A terceira entrega acadêmica, prevista para 19/10/2026, corresponde ao relatório e ao código da segunda versão do software. O enunciado exige servidor HTTP em Linux, C e Pthreads, com clientes concorrentes, conexões persistentes e mecanismos de qualidade de serviço (QoS) [1]. A primeira versão constitui a base de HTTP e concorrência; a segunda inclui controle de envio por IP e monitoramento.

O servidor aplica o ritmo de envio na camada de aplicação, acima do TCP. Isso não substitui o controle de congestionamento ou de fluxo do transporte. A implementação é um subconjunto didático de HTTP/1.1, com GET, HEAD e arquivos estáticos, e não uma implementação completa das RFCs [3, 4].

2. Metodologia e organização

O pacote mantém a primeira versão em V1 e a segunda em V2. A revisão de 19/10 foi feita sobre a V2 entregue pela equipe: inspeção do código e do relatório, comparação com os requisitos, reprodução de cenários e correção dos problemas identificados. Foram preservados o PCAP, o CSV e os registros originais em `evidencias/v2`. As novas evidências ficam em `evidencias/revisao-v2`, sem substituir os números históricos.

O relatório fornecido pela equipe registra acompanhamento por Trello e divisão de responsabilidades. Esses dados de gestão não foram verificados pela execução técnica da

revisão. O cronograma restante prevê completar a avaliação com as ferramentas exigidas e preparar a validação com o docente até 26/10, seguida dos slides até 03/12, conforme o enunciado.

3. Desenvolvimento e correções

3.1. Taxas, concorrência e admissão

A execução segue o formato `./server PORTA DIRETORIO VAZAO_MAX_KBPS [ARQUIVO_TAXAS]`. O arquivo auxiliar associa IPv4 e taxa inteira em kbps, usando a convenção decimal: 1 kbps = 1000 bits/s. Endereços não cadastrados usam 1000 kbps, desde que essa reserva caiba na capacidade informada.

Um mutex protege a tabela de IPs, suas conexões e a soma reservada. Se o IP já estiver ativo, a conexão divide a reserva existente; para um IP sem conexões ativas, a admissão exige $V_{reservada} + R_{IP} \leq V_{max}$. A última conexão libera a reserva. Uma recusa gera resposta 503. O limite de 128 conexões também protege os recursos do processo.

A revisão rejeita taxas zero, negativas, inválidas, maiores que 1.000.000.000 kbps, valores fora da representação numérica, IPv4 inválido, IP duplicado e campos excedentes. A capacidade passa pela mesma validação numérica. Comentários com # e linhas vazias são aceitos. Um arquivo explicitamente informado que não pode ser aberto interrompe a inicialização; quando apenas o arquivo padrão não existe, permanece a taxa padrão.

3.2. Envio e divisão da taxa

HTML é enviado sem limitação. Os demais objetos são enviados em blocos de até 8192 bytes. A taxa individual é recalculada a cada bloco por $R_c = R_{IP}/N$, com divisão em ponto flutuante. Isso evita truncamento e a imposição indevida de uma taxa mínima de 1 kbps para cada conexão. Após enviar n bytes, a espera nominal é

$$t_{espera} = \frac{8n}{1000 R_c} \quad \text{segundos.} \tag{1}$$

A espera usa `nanosleep`, fora do mutex e em fatias de até 100 ms, permitindo encerramento mesmo com taxas muito baixas. O controle admite uma rajada inicial de um bloco; o tempo de leitura e envio não é descontado da espera. Portanto, não garante uma taxa instantânea rígida em todas as janelas de tempo.

3.3. Painel e estimativas

Uma thread atualiza o painel aproximadamente a cada segundo. Para cada IP, mostra conexões, taxa reservada, taxa por conexão, estimativas e bytes de objetos enviados durante o atendimento atual. A coluna REAL agora usa o total de bytes de todas as conexões daquele IP na janela entre atualizações:

$$V_{IP} = \frac{8\,\Delta B_{IP}}{1000\,\Delta t} \quad \text{kbps,} \tag{2}$$

com Δt em segundos. O contador é reiniciado quando um IP volta a ser admitido após ficar sem conexões. A versão anterior mostrava a média de uma transferência individual em uma linha que representava o IP completo.

O intervalo entre a solicitação do HTML e a do primeiro objeto do mesmo IP é usado como estimativa didática de RTT, com resolução de milissegundos e mínimo de 1 ms. O painel mostra também metade desse intervalo e o tamanho do HTML em bits dividido pelo intervalo. Esses números incluem transmissão e processamento do cliente: não medem diretamente RTT TCP, atraso unidirecional ou capacidade do enlace. Navegações simultâneas do mesmo IP podem interferir na associação.

4. Avaliação experimental

4.1. Evidências originais da equipe

Os resultados abaixo são os registros fornecidos com a V2 original, anteriores às correções. O relatório original informa execução em Zorin OS, GCC 13.3.0, curl e tcpdump, na interface loopback. Essa configuração foi informada pela equipe e não corresponde necessariamente ao ambiente da revisão.

Table 1. Resultados originais, objeto de 1 MiB por transferência.

Cenário	IP	Status	Vazão útil (kbps)
Isolado	127.0.0.1	200	2001,2
Três IPs	127.0.0.1	200	2003,2
Três IPs	127.0.0.3	200	1503,4
Três IPs	127.0.0.4 (padrão)	200	1004,7
Admissão	127.0.0.2	503	–
Mesmo IP, conexão 1	127.0.0.1	200	1004,3
Mesmo IP, conexão 2	127.0.0.1	200	1004,3

A inspeção programática do PCAP original confirmou 1647 pacotes, 8 fluxos TCP, 8 SYN, 8 SYN-ACK, 14 requisições GET, 12 respostas 200 e 2 respostas 503, compatíveis com o relatório fornecido. Essa inspeção não foi feita na interface do Wireshark e não substitui a avaliação solicitada com essa ferramenta. O script original usa um objeto binário não referenciado no HTML; por isso, a revisão acrescenta o teste da página com as imagens efetivamente referenciadas.

4.2. Testes da versão corrigida

A revisão compilou o servidor com C11, otimização O2, Pthreads e opções `-Wall -Wextra -Wpedantic -Werror`. Os testes abrem sockets TCP reais no loopback. A suíte verifica configuração, inicialização, persistência, integridade, admissão, liberação de reservas, taxa padrão, divisão entre conexões e vazão agregada do painel. Os resultados detalhados e logs estão em `evidencias/revisao-v2`.

Os 17 testes foram aprovados. A configuração inválida foi rejeitada antes da abertura do serviço. No teste com imagens, foram recebidos 50.331.864 bytes de conteúdo em 4,6505 s, com vazão útil agregada de 86,58 Mbps. A persistência e os hashes SHA-256 das quatro transferências foram confirmados.

No teste separado de QoS, a taxa padrão de 1000 kbps resultou em 936,2 kbps. Duas conexões do IP com reserva de 2000 kbps atingiram 950,4 e 944,1 kbps. As amostras agregadas do painel com duas conexões ficaram entre 1964,1 e 2097,2 kbps,

em torno da reserva total de 2000 kbps. O teste de admissão observou 503 quando não havia reserva disponível e 200 após sua liberação.

O cenário de imagens usa dois clientes concorrentes do mesmo IP, com reserva de 128000 kbps e capacidade de 128000 kbps. Cada cliente solicita HTML, imagem1.bmp e imagem2.bmp na mesma conexão. Cada imagem possui 12.582.966 bytes, verificados por SHA-256. A taxa elevada foi escolhida para testar arquivos completos rapidamente; os testes separados de QoS usam taxas menores.

As medições são observações locais, sem caracterização de capacidade máxima ou intervalos de confiança. A vazão útil do lote usa bytes de corpo recebidos e tempo total até a conclusão dos dois clientes. Ela difere da vazão da interface, que pode incluir cabeçalhos, retransmissões e tráfego de fundo.

4.3. Pendências de ferramentas e redes físicas

Wireshark, tcpdump e IPTraf não estão instalados no ambiente da revisão. Não foram produzidas novas telas dessas ferramentas ou novas medições de IPTraf. Nenhum resultado foi inventado para preencher essa ausência. A avaliação exigida pelo enunciado permanece pendente até a execução no computador da equipe. Ethernet e Wi-Fi com hosts distintos também não foram avaliados; os vários endereços de loopback não representam hosts físicos.

O arquivo `V2/COMO-VALIDAR.md` e o script `V2/avaliar-ferramentas.sh` orientam a captura com tcpdump, o uso do Wireshark e a medição com IPTraf. O script verifica dependências e o processo de captura, preservando os logs. A equipe deve registrar a interface, os hosts, as unidades, a duração, o tráfego de fundo, retransmissões e, preferencialmente, três repetições. Até isso ocorrer, não se afirma cumprimento integral da avaliação.

5. Conclusões e próximos passos

A versão corrigida implementa os mecanismos centrais de QoS e corrige falhas de validação e de apresentação da vazão por IP. Os testes com arquivos completos acrescentam evidência de persistência e integridade das imagens da página. As taxas configuradas são reservas e alvos de envio de objetos; HTML, cabeçalhos e erros não entram no limitador e não se afirma um teto físico absoluto. Conexões ociosas participam da divisão enquanto permanecerem abertas.

Antes de submeter uma avaliação completa, a equipe deve executar e incorporar as evidências reais de Wireshark/IPTraf. Na evolução, convém usar prazos de envio cumulativos, melhorar a resolução temporal, separar sessões de navegação do mesmo IP e repetir os cenários em redes físicas. Todos os integrantes devem compreender as escolhas e conseguir demonstrar o código na validação com o docente.

6. Apoio de IA e disponibilidade

A V2 original declara apoio de Claude (Anthropic). A V1 original também registra apoio de IA. As correções, os novos testes e esta documentação tiveram apoio de OpenAI ChatGPT em 19/10/2026. Esses apoios estão registrados no código e README. A equipe deve revisar as alterações e executar os experimentos complementares. Não houve publicação automática no GitHub.

Referências

[1] Fundamentos e Avaliação de Redes de Computadores (2026). Trabalho de programação em rede: especificação do trabalho prático. 2026/2.

[2] Forouzan, B. A.; Mosharraf, F. (2013). Redes de computadores: uma abordagem top-down. AMGH.

[3] Fielding, R.; Nottingham, M.; Reschke, J. (2022). HTTP/1.1. RFC 9112. https://www.rfc-editor.org/rfc/rfc9112.

[4] Fielding, R.; Nottingham, M.; Reschke, J. (2022). HTTP Semantics. RFC 9110. https://www.rfc-editor.org/rfc/rfc9110.

[5] OpenAI (2026). ChatGPT. Apoio à revisão em 19/10/2026. https://chatgpt.com/. Anthropic (2026). Claude. Apoio declarado na V2 original.